

# MICRO-GRAPHIC SYSTEM SPECIFICATION

V.01-ALPHA // CORE DESIGN ARCHITECTURE // [REDACTED\_PROJECT\_ID]

## 01. CONCEPTUAL VISION

This application serves as a scalable, grid-based vector and typographic engine specifically engineered for "Micro-Graphics"—the high-density, technical information clusters found on industrial hardware, regulatory labels, and technical packaging. It seeks to unify the brutal efficiency of industrial design with the flexibility of generative programming.

**Intended Function:** A tool where the layout is an *emergent property* of the data itself, governed by a rigid set of industrial "physics" rules rather than manual placement.

## 02. FUNCTIONAL PRINCIPLES

- **Elastic Grid Architecture:** Users draw a bounding box; the system populates it with a coordinate grid that scales its "resolution" based on the available space.
- **Constraint-Based Packing:** Elements utilize a "Tetris-style" 2D bin packing logic inside horizontal partitions (the "Guillotine" method).
- **Atomic Integrity:** Text and icons are treated as "Genetic Species." Icons never stretch; text breaks only when the "Liquid" rule is toggled, otherwise preferring "Atomic" scaling or abbreviation.

## 03. ELEMENT TYPE LOGIC (THE THREE SPECIES)

Information is categorized into three primitive behaviors to ensure harmonious emergence:

- **Type A: The Spanner** (Logotypes/Headers). Forces a full-width row occupancy.

- **Type B: The Block** (Pictograms/Regulatory Icons). Clusters side-by-side in high-density groups.
- **Type C: The Data-Stream** (Specs/Voltages). Utilizes tabular column alignment for maximum legibility in small spaces.

## 04. TECH STACK & ENGINEERING SPECS

```
[ CORE_STACK ]
- FRONT_END: Web-based UI with interactive sidebar
- VECTOR_ENGINE: Paper.js (Path math and boolean geometry)
- FONT_ENGINE: Opentype.js (Direct metric analysis: X-height, cap-height, kerning)
- RENDERING: Scalable SVG or high-performance Canvas (Pixi.js)

[ UNIT_SYSTEM ]
- UNIT_ATOM (1U): Equal to the X-height of the primary typeface.
- STROKE_WEIGHT: Defined as a fraction of 1U (e.g., 0.125U).
- GUTTERS: Standardized relational spacing (e.g., 0.5U or 1U).
```

## 05. USER EXPERIENCE FLOW

1. **Initialization:** Define the "Canvas" by drawing a rectangle.
2. **Payload Entry:** User adds data elements to the Sidebar.
3. **Procedural Rendering:** The system "pours" the sidebar elements into the canvas, applying the "Guillotine" partition for rows and "Tetris" packing for clusters.
4. **Refinement:** Reordering sidebar items triggers a real-time recalculation of the grid occupancy.